SYSTEM DESIGN / INTERVIEW PLAYBOOK

The Complete Guide to Redis

Every internal, data structure, failure mode and coordination pattern you can be asked about in an HLD / system design interview — explained in plain words, with diagrams, real commands and model answers.

Redis to the metal

Event loop, encodings, expiry, RDB/AOF, replication, Sentinel, Cluster slots, memory.

Far beyond caching

Rate limiters, locks, leaderboards, queues, idempotency, HyperLogLog, Bloom, geo.

Where it lets you down

Async replication, a blocked event loop, hot slots, big keys, fork pauses — and what to say about each.

Command reference

The commands a working engineer knows by heart, with cost and purpose — plus the five that mark you as dangerous.

3 parts + command reference · 15 interview Q&A · 5 rate limiters

Revised August 2026

CONTENTS

What's inside

Read it front to back once to build the mental model. After that, Part 1 (internals), the command reference and Part 3 (interview answers) are the pages you revise the night before.

This volume is about Redis itself. Caching as a discipline — where to put a cache, the six patterns, invalidation, eviction policy and the classic failure modes — is a separate volume: *The Complete Guide to Caching*.

How to use this guide

Understand

Every section opens with a plain-English explanation before any jargon. If a sentence needs a diagram, there is one.

Say out loud

Grey “interview line” boxes give you the exact sentence that earns the point. Practise saying them.

Defend

Every choice lists its trade-off. Interviewers score you on the trade-off, not the choice.

A NOTE ON CODE

Code appears where the Redis logic *is* the answer and interviewers genuinely ask you to write it: the five rate limiters, the lock, the idempotency marker — and the command reference at the back. Everything else is explained in words, steps and diagrams, because interviews test **reasoning, not syntax**.

THE ONE THING TO INTERNALISE

Redis is a **single-threaded, in-memory data-structure server with asynchronous replication**. Almost every Redis answer follows from those three facts: one slow command blocks every client, server-side structures turn read-modify-write into one atomic step, and an acknowledged write can still be lost on failover. Say which of the three your design leans on.

Redis deep dive

PART 1

Redis is the default answer to "what would you cache with?" This part is why it is fast, what it stores, how it survives restarts, how it scales, and where it will let you down.

1.1 What Redis actually is

REmote **DI**ctionary **S**erver: an in-memory data structure server. The word that matters is **structure**. Memcached gives you a map from string to opaque bytes. Redis gives you server-side lists, sets, sorted sets, hashes, streams and bitmaps, each with atomic operations. That turns problems that would need read-modify-write round trips and application-level locking into a single atomic command.

> **INTERVIEW LINE**
>
> "I'd pick Redis over Memcached less for speed — they're comparable — and more because its data structures collapse multi-step application logic into single atomic operations. A leaderboard is a sorted set, a rate limiter is an `INCR` with an expiry, and a dedupe check is a set membership test. With Memcached each of those needs application-side coordination that is hard to get right under concurrency."

1.2 The threading model — why single-threaded is fast

Redis executes all commands on **one thread**, in a single event loop. This surprises people, so be ready to explain why it is a feature:

- **It is not CPU-bound.** A hash lookup in RAM takes nanoseconds; the bottleneck is network I/O and memory bandwidth, not computation. Adding cores would not help the thing that is actually slow.
- **No locks, no contention, no race conditions.** Every command is atomic *by construction*. This is why `INCR`, `SETNX` and `LPUSH` are safe under any level of concurrency without you doing anything.
- **No context switching** between threads, and no cache-line ping-ponging between cores.
- **I/O multiplexing** (epoll/kqueue) lets that one thread service tens of thousands of connections.

Modern Redis is not purely single-threaded any more, and knowing the nuance is worth points:

io-threads	Since 6.0, reading requests from and writing replies to sockets can be parallelised across threads. **Command execution stays single-threaded.**
Background threads	`UNLINK`, `FLUSHALL ASYNC`, lazy-free of big objects, AOF fsync and RDB saves happen off the main thread (or in a forked child).
Redis 8	Continues to push toward more parallel I/O and per-slot processing while preserving single-threaded command semantics.

THE CONSEQUENCE YOU MUST STATE

Because one thread runs every command, **a single slow command blocks every other client.** `KEYS *`, `FLUSHALL`, `SMEMBERS` on a 10-million-member set, a Lua script with a loop, or `DEL` on a huge key will freeze the entire server for its full duration. This is the root cause of both the big-key problem and most mysterious Redis latency incidents. Keep every operation O(1) or O(log N), and use `SCAN`-family cursors for anything that iterates.

1.3 Data types — and what each one is *for*

Type	Key commands	Use it for	Complexity notes
String	`SET GET INCR SETEX APPEND SETRANGE`	Cached objects/JSON, counters, flags, locks	O(1). Max 512 MB. `INCR` is atomic
Hash	`HSET HGET HGETALL HINCRBY HSCAN`	Objects where you read/write single fields; session data	O(1) per field. Avoid `HGETALL` on large hashes
List	`LPUSH RPOP LRANGE LTRIM BLPOP`	Queues, activity feeds, recent-N (with `LTRIM`)	O(1) at the ends, O(N) in the middle. `BLPOP` blocks
Set	`SADD SISMEMBER SINTER SCARD SPOP`	Unique visitors, tags, dedupe, social graph intersections	O(1) membership; `SINTER` is O(N×M) — careful
Sorted set	`ZADD ZRANGE ZRANK ZINCRBY ZRANGEBYSCORE`	**Leaderboards, priority queues, sliding-window rate limits, time-series indexes**	O(log N). The most powerful type — know it well
Bitmap	`SETBIT GETBIT BITCOUNT BITOP`	Daily-active-users, per-user boolean features	1 bit/user → 10 M users in 1.2 MB
HyperLogLog	`PFADD PFCOUNT PFMERGE`	Approximate unique counts at massive scale	**12 KB for billions of items**, ~0.81% error. Cannot list members
Geospatial	`GEOADD GEOSEARCH GEODIST`	"Drivers near me", store locators	Sorted set + geohash under the hood
Stream	`XADD XREADGROUP XACK XAUTOCLAIM`	Event log with consumer groups — a durable Kafka-lite	Append-only, persistent, replayable, at-least-once
Bitfield	`BITFIELD`	Packed integer counters in one string	Extreme memory efficiency
JSON *	`JSON.SET JSON.GET JSON.NUMINCRBY`	Document storage with sub-path access	Module (RedisJSON); core in Redis 8
Vector *	`FT.CREATE … VECTOR`, `FT.SEARCH`	Semantic caching, RAG, similarity search	Redis 8 native. The 2026 growth area
Bloom *	`BF.ADD BF.EXISTS CF.ADD`	Cache-penetration guard, dedupe at scale	RedisBloom module

1.4 Internal encodings (the “have you read the source?” question)

Redis silently switches each type's internal representation based on size, trading CPU for memory on small objects. A small hash is stored as a flat array that is scanned linearly — because for 20 elements that is faster *and* far smaller than a hash table.

Type	Compact encoding	Promotes to	Threshold config
String	`int` / `embstr` (≤44 B)	`raw` SDS	—
Hash	`listpack`	`hashtable`	`hash-max-listpack-entries` 128 / `-value` 64
List	`listpack`	`quicklist` (linked listpacks)	`list-max-listpack-size` 128
Set	`intset` / `listpack`	`hashtable`	`set-max-intset-entries` 512
Sorted set	`listpack`	`skiplist` + hashtable	`zset-max-listpack-entries` 128

Practical consequence: **many small hashes are dramatically cheaper than one huge hash**. Instagram's well-known memory-optimisation post used exactly this — bucketing IDs into thousands of small hashes cut memory by roughly 5×. A sorted set uses a **skiplist** once large, which is why `ZRANK` is O(log N); the parallel hashtable is what makes `ZSCORE` O(1).

1.5 How keys actually expire

A common trick question: “if a key's TTL passes, is memory freed at that instant?” No. Redis uses two mechanisms:

Lazy expiration

When a key is accessed, Redis checks its TTL and deletes it if expired. Zero cost for keys nobody touches — but a key that expired and is never read again would occupy memory forever.

Active expiration

Ten times per second, Redis samples 20 random keys from the expires table, deletes those that are expired, and if more than 25% were expired it immediately repeats. Probabilistic cleanup that adapts to how much garbage exists.

Consequences worth stating: expired-but-not-yet-collected keys still count toward `maxmemory`; `DBSIZE` can overcount; and on a replica, **keys are not expired independently** — the primary sends an explicit `DEL`, so a replica may briefly return a logically-expired key (modern versions hide this from reads, but the replication mechanism is what interviewers ask about).

1.6 Persistence — RDB vs AOF

Two mechanisms, two failure profiles

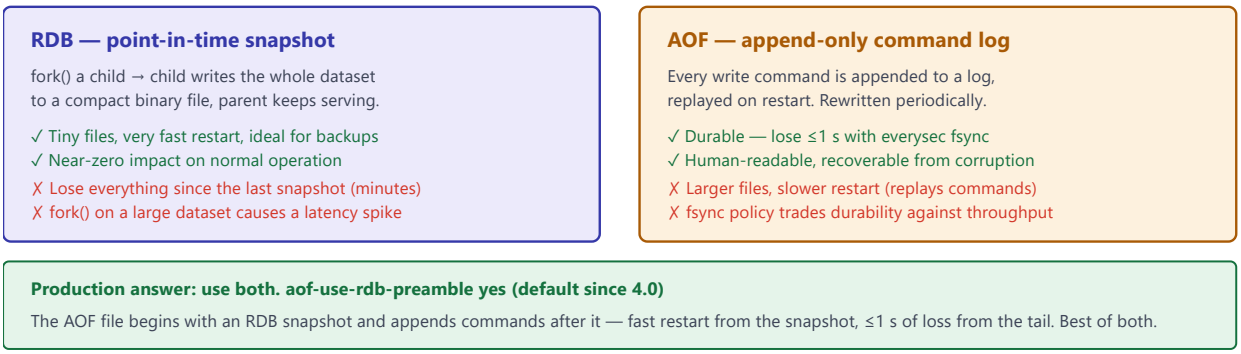


Figure 1.1 — *RDB, AOF, and the hybrid that is the real-world default.*

appendfsync always	fsync on every write. Safest, dramatically slower. Rarely used.
appendfsync everysec	**The default and the right answer.** At most 1 second of writes lost.
appendfsync no	Let the OS decide (~30 s of exposure). Fastest, least safe.
save 900 1	RDB trigger: snapshot if ≥1 key changed in 900 s. Multiple rules stack.

> **SHOULD A PURE CACHE HAVE PERSISTENCE?**
>
> The instinctive answer is no — it's just a cache, let it rebuild. The better answer is operational: if losing the cache means a 100× load spike at your database, then **restart time is an availability concern**, and RDB lets a restarted node come back warm in seconds instead of cold. Also note the memory cost: `fork()` uses copy-on-write, so a snapshot on a write-heavy instance can transiently need **up to 2× the memory**. That is why you should not run Redis at more than ~50–60% of the box's RAM if you snapshot.

1.7 Replication

One primary, N replicas, **asynchronous** by default. The primary does not wait for replicas before acknowledging a write.

- **Full sync:** primary forks, produces an RDB, ships it, then streams the backlog of commands accumulated meanwhile.
- **Partial resync:** after a brief disconnect, the replica asks to resume from an offset in the primary's replication backlog buffer — no full RDB transfer.
- **Replicas are read-only** by default and serve read traffic, which is the standard read-scaling lever.
- **Replication lag** means a read from a replica can return stale data. If a user writes and immediately reads, they may not see their own write — route read-your-writes traffic to the primary.
- `WAIT numreplicas timeout` blocks until N replicas have acknowledged. This gives you stronger durability per-write, but **it is not consensus** — it cannot prevent all lost writes under failover.

REDIS IS NOT A CP SYSTEM — SAY THIS IF DURABILITY COMES UP

Because replication is asynchronous, a primary can acknowledge a write, fail before replicating it, and have a replica promoted that never saw it. **The acknowledged write is silently lost.** Redis chooses availability and latency over strict consistency. This is fine for a cache and it is the reason you should be careful about treating Redis as a source of truth — and it is the foundation of the Redlock critique in §2.2.

1.8 High availability — Sentinel

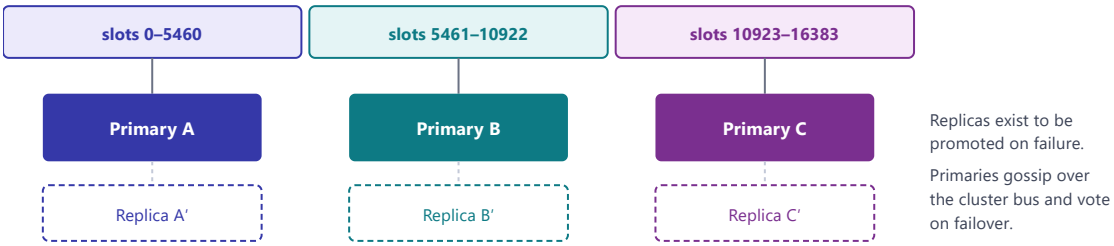
Sentinel is a separate set of processes that monitor a primary, agree via quorum that it is down (*subjectively down → objectively down*), elect a leader Sentinel, promote a replica, and reconfigure the others. Clients ask Sentinel for the current primary address rather than hard-coding it.

- Run an **odd number, at least 3**, on separate failure domains — quorum requires a majority.
- Sentinel provides **automatic failover, not sharding**. All data still fits on one primary.
- Split-brain is mitigated with `min-replicas-to-write`: a primary that cannot see enough replicas stops accepting writes.

1.9 Redis Cluster — horizontal scale

When data or write throughput exceeds one machine, you shard. Redis Cluster does this natively with **16,384 hash slots**: `slot = CRC16(key) mod 16384`, and each primary owns a contiguous range of slots.

Redis Cluster: 16,384 slots distributed over primaries



Why 16,384 fixed slots instead of hashing directly to nodes?

Because the mapping key→slot never changes; only slot→node does. Rebalancing means moving whole slots between nodes, not rehashing the keyspace. Adding a 4th node just reassigns ~1/4 of the slots. It is consistent hashing with a fixed, cheap-to-gossip bitmap — 16,384 bits is 2 KB, small enough to include in every heartbeat between nodes.

During a live migration: the old node replies MOVED (permanent, update your map) or ASK (temporary, this one key only).

Figure 1.2 — *Cluster topology. Clients cache the slot map and route directly — there is no proxy in the data path.*

THE CLUSTER LIMITATION THAT SHAPES YOUR DESIGN

Multi-key operations only work if all keys are in the same slot. `MGET a b c`, `SINTER`, transactions and Lua scripts across keys will fail with `CROSSSLOT` otherwise. The escape hatch is a **hash tag**: only the part inside `{}` is hashed, so `user:{42}:profile` and `user:{42}:sessions` are guaranteed to land together. Use tags to co-locate related keys — but remember that over-using them recreates hot shards.

1.10 Memory management

used_memory	What Redis's allocator reports it is using for data.
used_memory_rss	What the OS says the process holds. RSS > used_memory means **fragmentation**.
mem_fragmentation_ratio	rss / used_memory. ~1.0–1.5 is healthy; >1.5 means wasted memory (enable `activedefrag`); **<1.0 means Redis is swapping to disk** — a latency emergency.
Overhead	Each key costs roughly 50–100 bytes beyond its value (dict entry, SDS header, expire entry, pointers). **A million tiny keys can cost more in overhead than in data.** This is the argument for bucketing into hashes.

The diagnostic commands for all of this — `INFO memory`, `MEMORY DOCTOR`, `--bigkeys`, `--hotkeys`, `--latency-history` — are collected in the command reference at the end of this guide.

1.11 Atomicity: transactions, Lua, and Functions

Mechanism	What it gives you	What it does not
MULTI / EXEC	Commands queued and executed together, with no other client interleaved	**No rollback.** If one command fails, the others still apply. It is isolation, not atomicity-with-recovery
WATCH	Optimistic locking — `EXEC` aborts if a watched key changed	Requires a client-side retry loop
Lua (`EVAL`)	Arbitrary read-modify-write logic executed atomically on the server, in one round trip	Blocks the single thread for its whole duration — keep scripts tiny
Functions (7.0+)	Lua libraries registered server-side and persisted/replicated, rather than shipped per call	Same blocking caveat
Pipelining	Send N commands without waiting for each reply — amortises RTT, often 5–10× throughput	**Not atomic.** Purely a network optimisation. Interviewers love this distinction

> **WHY LUA EXISTS, IN ONE EXAMPLE**
>
> "Delete this lock, but *only if I still own it*" is two operations — a read and a conditional delete. Between them, another client can legitimately acquire the lock, and your delete would then remove **their** lock. There is no single Redis command for "compare and delete," so you send both steps as one Lua script: Redis runs it start-to-finish with nothing interleaved. **That is the whole purpose of Lua here — turning a read-then-write sequence into one indivisible operation.** The same reasoning drives the rate-limiter scripts in Part 2.

1.12 Pub/Sub vs Streams

Pub/Sub — fire and forget

`PUBLISH` / `SUBSCRIBE`. Messages go to whoever is connected *right now*. **No persistence, no replay, no acknowledgement.** A subscriber that is down misses everything permanently.

Right for: cache invalidation broadcasts, live presence, ephemeral notifications — things where missing a message is survivable.

Streams — durable log

`XADD` / `XREADGROUP`. An append-only log with consumer groups, per-consumer offsets, explicit `XACK`, and a pending-entries list so unacknowledged work can be reclaimed with `XAUTOCLAIM`.

Right for: job queues, event sourcing, anything needing at-least-once delivery. Kafka-like semantics without operating Kafka.

1.13 Client-side caching (RESP3 tracking)

The feature that makes the L1+L2 topology *correct* rather than merely bounded-stale. With `CLIENT TRACKING ON`, Redis records which keys each client has read and pushes an invalidation message the moment any of them changes.

There are two modes, and the trade-off between them is the interesting part:

- **Default mode** — Redis remembers exactly which keys each client has read, and notifies only the clients that care. Precise, but the server spends memory tracking it.
- **Broadcast mode** — clients subscribe to key *prefixes* instead. Redis keeps no per-client state, but sends more invalidations than strictly necessary.

Mentioning this is a strong differentiator on any “how do you keep local caches coherent?” question, because it converts L1 coherence from a guess (short TTL) into a protocol guarantee.

1.14 Redis vs the alternatives

Dimension	Redis	Memcached	Valkey
Data types	**10+ rich structures**	**Strings only**	**Same as Redis 7.2**
Threading	Single-threaded exec + I/O threads	**Truly multi-threaded**	Multi-threaded I/O; strong per-core scaling
Raw GET/SET throughput	Very high	**~20–25% higher at high concurrency**	Slightly ahead of Redis 8 in recent benchmarks
Persistence	**RDB + AOF**	**None**	**RDB + AOF**
Replication / HA	**Replicas, Sentinel, Cluster**	**None built in**	**Same**
Memory per small item	Higher overhead	**Leaner; slab allocator**	Comparable to Redis
Eviction	8 configurable policies	Slab LRU	Same as Redis
Licence	RSALv2/SSPL, then AGPLv3 (v8)	**BSD**	**BSD, Linux Foundation**
Pick it when	**Almost always** — structures, persistence and HA make it the default	Pure ephemeral string cache, multi-core box, want the simplest possible thing	You want Redis semantics with a permissive licence / cloud-vendor backing

Context worth knowing: Redis changed from BSD to a source-available licence in 2024, prompting AWS, Google and others to fork Redis 7.2.4 into **Valkey** under the Linux Foundation. Redis 8 later adopted AGPLv3 alongside its existing licences. Valkey is command-compatible, so most "Redis" design answers apply unchanged. Also worth naming: **Dragonfly** (multi-threaded, Redis-compatible, vertical scaling) and **ElastiCache / MemoryDB** (AWS managed; MemoryDB adds a durable multi-AZ transaction log, making it the CP-flavoured option).

> **INTERVIEW LINE**
>
> "I'd default to Redis. Memcached is genuinely faster for pure string GET/SET on a multi-core box, but the moment I need a sorted set for a leaderboard, atomic rate limiting, persistence so a restart doesn't cold-start me, or built-in failover, I'd be rebuilding those in the application. Valkey is a drop-in if licensing matters to the org."

Redis beyond caching

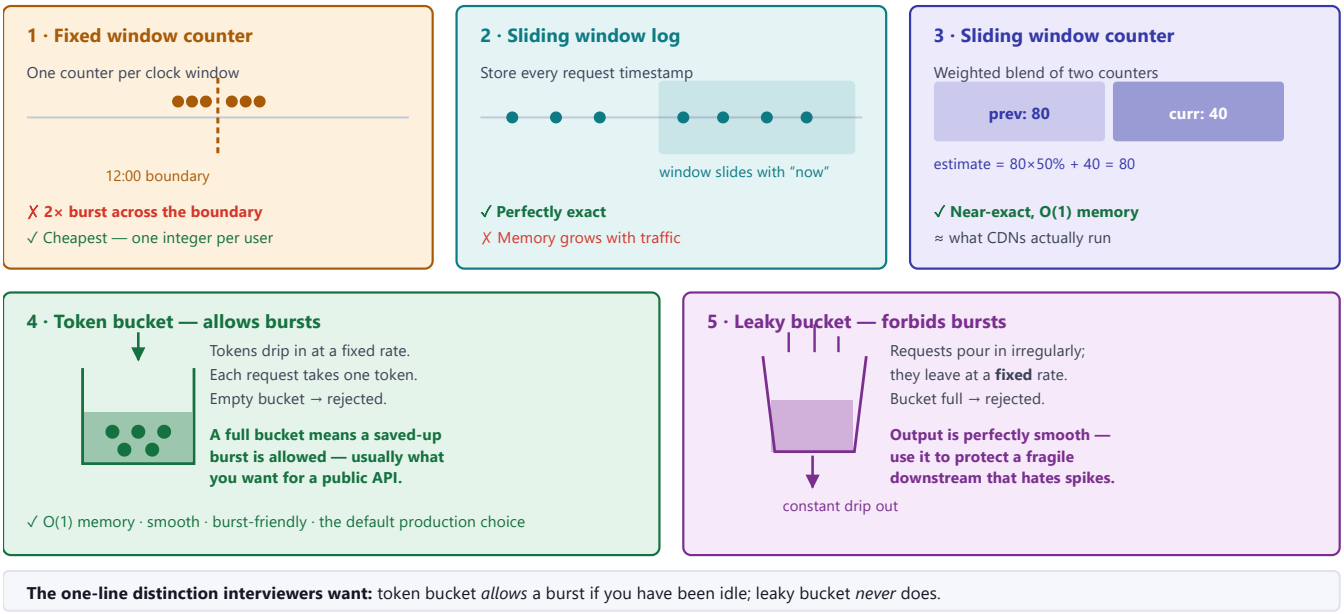
PART 2

Redis shows up in system design answers far more often as a coordination primitive than as a cache. These are the patterns worth having at your fingertips.

2.1 Rate limiting — all five algorithms

"Design a rate limiter" is one of the most frequently asked Redis questions, and interviewers usually want **all five algorithms, in order, with the flaw of each**. This section is written to be memorised. Every implementation below is Lua, for one reason: a rate-limit check is a *read-then-write* sequence, and if two requests interleave between the read and the write, both get allowed. Lua makes the whole check one atomic step.

The five algorithms at a glance



The one-line distinction interviewers want: token bucket *allows* a burst if you have been idle; leaky bucket *never* does.

Figure 2.1 — *The five algorithms. Learn them in this order — each one fixes the flaw of the one before it.*

1 · Fixed window counter

Idea: chop time into fixed windows (say each calendar minute) and keep one counter per user per window.
Redis structure: a String, keyed by user + window number.

LUA (FIXED WINDOW)

```lua
-- KEYS[1] = "rl:user42:28918350"   (user + floor(now/window))
-- ARGV[1] = window seconds        ARGV[2] = limit
local count = redis.call("INCR", KEYS[1])
if count == 1 then
    redis.call("EXPIRE", KEYS[1], ARGV[1])   -- set TTL only on the first request
end
return count <= tonumber(ARGV[2]) and 1 or 0
```

THE FLAW YOU MUST NAME

With a limit of 100/minute, a user can send **100 requests at 11:59:59 and 100 more at 12:00:00** — 200 requests in one second, double the intended rate. The window boundary is a free reset.

Also note the subtle bug the Lua fixes: doing `INCR` and `EXPIRE` as two separate round trips means a crash in between leaves a counter **with no TTL that never resets** — the user is locked out forever.

2 · Sliding window log

Idea: store the timestamp of every request, discard the ones older than the window, and count what's left. **Redis structure:** a Sorted Set, scored by timestamp.

LUA (SLIDING LOG)

```lua
-- ARGV: 1=now(ms)  2=window(ms)  3=limit  4=unique request id
local now, window, limit = tonumber(ARGV[1]), tonumber(ARGV[2]), tonumber(ARGV[3])

redis.call("ZREMRANGEBYSCORE", KEYS[1], 0, now - window)   -- drop what fell out

if redis.call("ZCARD", KEYS[1]) < limit then
    redis.call("ZADD", KEYS[1], now, now .. ":" .. ARGV[4])  -- id keeps members unique
    redis.call("PEXPIRE", KEYS[1], window)
    return 1
end
return 0
```

Perfectly accurate — there is no boundary to exploit, because the window truly slides. **The cost:** one sorted-set member per request. A limit of 10,000/hour per user means 10,000 stored timestamps per user. Great for low limits (5 login attempts per minute), far too expensive for high ones.

3 · Sliding window counter

Idea: the compromise. Keep two fixed-window counters — current and previous — and **weight the previous one by how much of it still overlaps** the sliding window.

THE FORMULA — WORTH MEMORISING EXACTLY

$$\text{estimate} = \text{previous count} \times (1 - \text{elapsed fraction}) + \text{current count}$$

Example. Limit 100/min. It is 12:00:30, so we are 50% through the current window. The 11:59 window saw 80 requests; the 12:00 window has seen 40 so far.
estimate = 80 × (1 − 0.5) + 40 = **80** → under 100, so **allow**.

LUA (SLIDING COUNTER)

```lua
-- KEYS[1] = current window key    KEYS[2] = previous window key
-- ARGV: 1=limit  2=window(s)  3=elapsed fraction of current window (0..1)
local limit, window, elapsed = tonumber(ARGV[1]), tonumber(ARGV[2]), tonumber(ARGV[3])

local curr = tonumber(redis.call("GET", KEYS[1])) or 0
local prev = tonumber(redis.call("GET", KEYS[2])) or 0

local estimate = prev * (1 - elapsed) + curr        -- the weighted blend

if estimate < limit then
    redis.call("INCR", KEYS[1])
    redis.call("EXPIRE", KEYS[1], window * 2)         -- keep it alive as "previous"
    return 1
end
return 0
```

Two integers per user, no boundary exploit, accuracy within a fraction of a percent in practice. It assumes requests were spread evenly through the previous window, which is why it is an approximation — but that assumption is close enough that this is what most CDNs and API gateways actually ship. **If you can only remember one "smart" algorithm, remember this formula.**

4 · Token bucket

Idea: a bucket holds up to *capacity* tokens and refills at *rate* tokens per second. Each request spends one token; no tokens means rejected. **Redis structure:** a Hash holding `tokens` and the `last refill timestamp`.

The trick that makes it O(1): **you never actually run a refill timer.** You lazily compute how many tokens *would* have accumulated since the last request, capped at capacity.

LUA (TOKEN BUCKET)

```lua
-- ARGV: 1=rate (tokens/sec)  2=capacity  3=now (seconds, fractional)
local rate, capacity, now = tonumber(ARGV[1]), tonumber(ARGV[2]), tonumber(ARGV[3])

local b      = redis.call("HMGET", KEYS[1], "tokens", "ts")
local tokens = tonumber(b[1]) or capacity      -- new user starts full
local ts     = tonumber(b[2]) or now

tokens = math.min(capacity, tokens + (now - ts) * rate)   -- lazy refill

local allowed = tokens >= 1
if allowed then tokens = tokens - 1 end

redis.call("HSET", KEYS[1], "tokens", tokens, "ts", now)
redis.call("EXPIRE", KEYS[1], math.ceil(capacity / rate) * 2)
return allowed and 1 or 0
```

Why it is the usual production answer: O(1) memory regardless of traffic, and it **rewards idle users** — someone who hasn't called in a while has a full bucket and can burst, which is almost always the behaviour you want from a public API. AWS, Stripe and most API gateways use this shape.

5 · Leaky bucket

Idea: requests pour into a bucket that drains at a constant rate. If the bucket overflows, reject. It is the mirror image of token bucket — and the difference is the whole point.

LUA (LEAKY BUCKET)

```lua
-- ARGV: 1=leak rate (per sec)  2=capacity  3=now (seconds)
local rate, capacity, now = tonumber(ARGV[1]), tonumber(ARGV[2]), tonumber(ARGV[3])

local b     = redis.call("HMGET", KEYS[1], "water", "ts")
local water = tonumber(b[1]) or 0             -- new user starts empty
local ts    = tonumber(b[2]) or now

water = math.max(0, water - (now - ts) * rate)      -- lazy leak

local allowed = (water + 1) <= capacity
if allowed then water = water + 1 end

redis.call("HSET", KEYS[1], "water", water, "ts", now)
redis.call("EXPIRE", KEYS[1], math.ceil(capacity / rate) * 2)
return allowed and 1 or 0
```

> **TOKEN BUCKET VS LEAKY BUCKET — THE QUESTION BEHIND THE QUESTION**
>
> The code looks nearly identical (one counts up, one counts down), so interviewers ask about the **behavioural** difference:
>
> **Token bucket** lets an idle user spend saved-up tokens all at once — traffic out is *bursty*. Use it when bursts are acceptable and you're protecting against sustained abuse.
> **Leaky bucket** drains at a fixed rate no matter what — traffic out is *perfectly smooth*. Use it when the thing downstream cannot tolerate spikes at all: a payment gateway, a legacy SOAP service, a third-party API with a hard per-second cap.

Comparison — the table to memorise

Algorithm	Redis type	Memory	Accuracy	Bursts?	Pick it when
Fixed window	String	**O(1)**	**Poor**	Yes, 2× at edges	Rough protection is enough and simplicity wins
Sliding log	Sorted Set	**O(requests)**	**Exact**	**No**	Low limits where exactness matters — login attempts, OTP sends
Sliding counter	2 Strings	**O(1)**	**~99%**	**No**	**Best accuracy-per-byte.** The general-purpose default
Token bucket	Hash	**O(1)**	**Exact**	**Yes, controlled**	**Public APIs.** Rewards idle users with burst capacity
Leaky bucket	Hash	**O(1)**	**Exact**	**Never**	Protecting a fragile downstream that needs smooth input

THREE FOLLOW-UPS THAT ALWAYS COME NEXT

"What if you have 50 API servers?" — the counter must be central, which is exactly why it lives in Redis rather than in each server's memory. If Redis round trips become the bottleneck, do **two-tier limiting**: a local per-server limiter for the common case, syncing to Redis periodically for the global view. You trade a little precision for a lot of latency.

"What if Redis goes down?" — decide deliberately: **fail open** (allow everything — protects user experience, risks abuse) or **fail closed** (reject everything — protects the backend, causes an outage). Most public APIs fail open for rate limiting and fail closed for anything security-critical.

"What do you return to the caller?" — HTTP **429 Too Many Requests**, plus `Retry-After` and the `X-RateLimit-Limit / -Remaining / -Reset` headers. Mentioning the headers unprompted signals you've built one rather than only read about them.

2.2 Distributed locks — and why to be careful

JAVA

```java
// Acquire: atomic set-if-not-exists WITH an expiry. Never do SETNX then EXPIRE —
// a crash between them leaves a lock nobody can release.
String token = UUID.randomUUID().toString();
boolean got = "OK".equals(
    redis.set("lock:order:42", token, SetParams.setParams().nx().px(10_000)));

// Release: compare-and-delete via Lua (see below). A plain DEL could delete
// a lock that someone else acquired after yours timed out.
redis.eval(RELEASE_SCRIPT, List.of("lock:order:42"), List.of(token));
```

REDLOCK AND THE KLEPPMANN CRITIQUE — KNOW BOTH SIDES

Redlock is the multi-node algorithm: acquire the lock on a majority of N independent Redis primaries within a time bound, and you hold it. Martin Kleppmann's 2016 analysis argued it is unsafe for correctness, and Redis's creator Salvatore Sanfilippo published a rebuttal. The core objections:

- **It assumes bounded clock drift, bounded network delay and bounded process pauses.** A GC pause or an NTP jump can let two clients believe they hold the lock simultaneously.
- **No fencing tokens.** A correct lock should hand out a monotonically increasing number that the protected resource checks and rejects if stale. Redlock does not, so a delayed writer from an expired lock can still corrupt state.

The answer that scores: "For *efficiency* — avoiding duplicate work like a cache rebuild — a simple single-instance Redis lock is fine, because the worst case is wasted effort. For *correctness* — where two holders would corrupt data or double-charge a customer — I wouldn't rely on Redis. I'd use a consensus system like ZooKeeper or etcd, or push the invariant into the database with a conditional update or unique constraint, which gives me fencing for free."

2.3 Leaderboards — the sorted set showcase

```redis-cli
ZADD leaderboard 4820 "player:17"          # O(log N) insert/update
ZINCRBY leaderboard 50 "player:17"          # atomic score bump
ZREVRANGE leaderboard 0 9 WITHSCORES       # top 10 — O(log N + 10)
ZREVRANK leaderboard "player:17"           # this player's rank — O(log N)
ZCOUNT leaderboard 1000 2000               # how many in a score band
```

The killer feature is `ZREVRANK`: **exact rank among millions of players in O(log N).** Doing that in SQL means `COUNT(*) WHERE score > x` — a full index scan per query.

Interview follow-ups: *Time-windowed leaderboards* → one ZSET per period (`lb:2026-08`) with a TTL, and `ZUNIONSTORE` to roll up. *Ties* → encode a timestamp into the low bits of the score so earlier achievers rank higher. *Too big for one node* → shard by score range, or keep only the top-N globally and approximate ranks below that.

2.4 Sessions

```redis-cli
HSET session:abc123 user_id 42 role admin csrf "tok"
EXPIRE session:abc123 1800            # sliding: re-EXPIRE on every request
```

Why Redis rather than sticky sessions: any app server can serve any user, so you can deploy, autoscale and lose instances freely. TTL gives you session expiry for free. Use a Hash so you can read one field without deserialising the whole session. **Caveat:** sessions are not regenerable from anywhere else — if you lose Redis, every user is logged out. Enable persistence and replication, and set the eviction policy to `volatile-lru` so a memory spike cannot silently evict live sessions.

2.5 Idempotency & deduplication

```java
// Payment API: the client sends an Idempotency-Key header.
// SET NX is the atomic "have I seen this before?" test.
if (!"OK".equals(redis.set("idem:" + idemKey, "processing",
        SetParams.setParams().nx().ex(86400)))) {
    return fetchPreviousResult(idemKey);   // duplicate — return the stored response
}
Result r = charge(request);
redis.setex("idem:" + idemKey, 86400, serialize(r));
return r;
```

At extreme scale, swap the per-key entry for a **Bloom filter** or **HyperLogLog** when approximate dedupe is acceptable. Note the honest caveat: because Redis replication is async (§1.7), a failover between the `SET NX` and the charge can lose the marker. For real money, the idempotency key belongs in the database as a unique constraint — Redis is the fast path, not the guarantee.

2.6 Counting at scale

Need	Structure	Cost
Exact count	`INCR` on a String	O(1), 8 bytes. Shard it if hot
Unique visitors (approx.)	`PFADD` / `PFCOUNT`	**12 KB for billions**, 0.81% error. `PFMERGE` combines days into a month
Per-user booleans	`SETBIT` / `BITCOUNT`	1 bit per user. 10 M users = 1.2 MB. `BITOP AND` gives retention cohorts
Exact unique members	`SADD` / `SCARD`	Full member storage — only when you must enumerate them
Top-K	`TOPK.*` (RedisBloom)	Approximate heavy hitters in fixed memory

2.7 Queues and delayed jobs

REDIS-CLI

```
# Simple work queue — blocking pop, no polling
LPUSH jobs '{"id":1}'
BRPOP jobs 0

# Reliable variant: atomically move to a processing list so a crashed
# worker's job isn't lost. Reaper returns stale items to `jobs`.
BLMOVE jobs processing RIGHT LEFT 0

# Delayed / scheduled jobs — sorted set scored by execute-at time
ZADD delayed 1755100000 "job:88"
ZRANGEBYSCORE delayed 0 <now> LIMIT 0 100   # poll due jobs

# Production choice: Streams with consumer groups (acks + replay)
XADD events * type order_placed id 42
XREADGROUP GROUP workers w1 COUNT 10 STREAMS events >
XACK events workers 1755100000-0
```

> **SAY THIS BEFORE AN INTERVIEWER SAYS IT TO YOU**
>
> Redis makes a fine queue for moderate volume and short-lived jobs, and Streams give you real acknowledgement semantics. But if you need long retention, replay from arbitrary offsets, high fan-out to many independent consumer groups, or strict ordering guarantees across partitions at very high throughput — **use Kafka**. Volunteering the boundary of the tool shows judgement.

2.8 Geospatial

```redis-cli
GEOADD drivers 77.5946 12.9716 "driver:7"
GEOSEARCH drivers FROMLONLAT 77.60 12.97 BYRADIUS 3 km ASC COUNT 10
GEODIST drivers "driver:7" "driver:9" km
```

Internally a sorted set scored by 52-bit geohash — which is why it inherits O(log N) behaviour. The standard answer for the proximity-search step of an Uber / Yelp / food-delivery design.

2.9 Semantic caching (the 2026 addition)

Worth a sentence if the interview touches AI systems. Classic caching keys on *exact* equality, which fails for natural language — "how do I reset my password" and "password reset steps" are the same question with different bytes. **Semantic caching** embeds the query as a vector, does a similarity search over cached query/response pairs, and returns the cached answer if similarity exceeds a threshold. Redis 8's native vector search makes Redis a common choice for this, alongside caching embeddings themselves and LLM tool-call results. The trade-off is a new failure mode: **a too-loose threshold returns a confidently wrong cached answer.**

The interview itself

PART 3

Knowing the material is necessary but not sufficient. This part is about *sequencing* — the order in which you say things, so the interviewer hears a designer rather than a list of memorised terms.

3.1 A framework that works every time

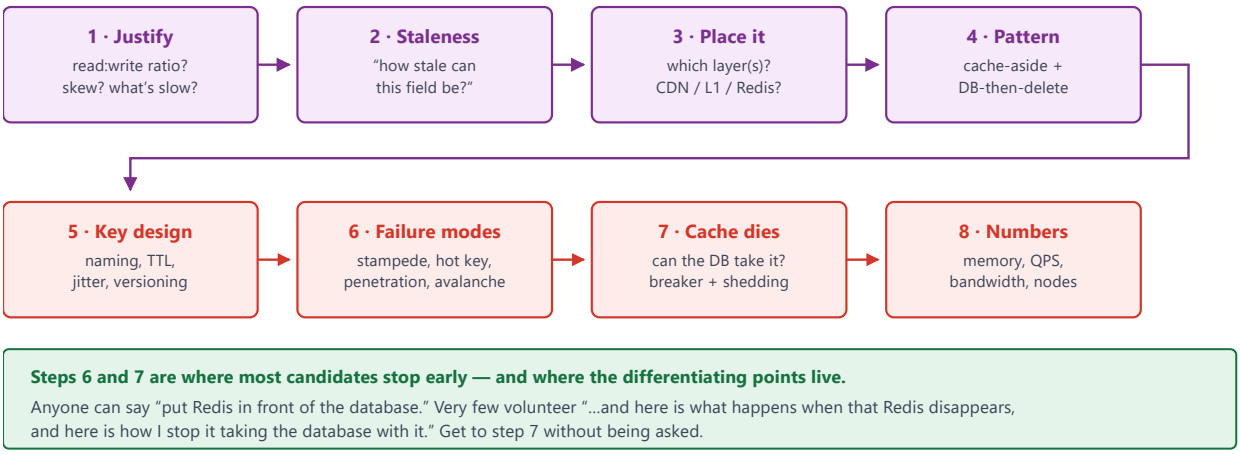


Figure 3.1 — *Eight steps. Steps 1–2 take twenty seconds and prevent most wrong turns.*

3.2 Key naming — a small thing that signals experience

The convention is a colon-delimited hierarchy: `app : entity : id : field : version`

Key	Verdict	Why
`user:1042:profile`	**Good**	Scannable, obvious owner, groups cleanly
`product:88:price:v3`	**Good**	The version lets you invalidate an entire generation at once
`lb:global:2026-08`	**Good**	Time-bucketed — the whole month expires by itself
`u1042`	**Bad**	No namespace — unsearchable, collides, meaningless in a dump
`"User Profile 1042"`	**Bad**	Spaces and unstable formatting; easy to generate two variants of the same key
`user:1042` holding 4 MB	**Bad**	A big key — blocks the single-threaded event loop for every other client

Mention: a consistent prefix lets you `SCAN MATCH product:*` for debugging, keeps metrics per-namespace, and lets you migrate one entity type at a time. Include a schema version in the prefix (`v3:product:88`) so a serialisation change is a cache miss rather than a deserialisation crash — **this is a genuinely under-mentioned production detail.**

3.3 Fifteen questions with model answers

Q1 Why is Redis fast if it is single-threaded?

Because it isn't CPU-bound — everything is in RAM, so the bottleneck is I/O, not computation. One thread means no locks, no contention, no context switching, and every command is atomic by construction. It uses epoll-based I/O multiplexing to serve tens of thousands of connections. Since 6.0, socket I/O can be threaded, but command execution remains single-threaded.

Always add the consequence: one slow command (`KEYS *`, a big-key `DEL`, a long Lua script) blocks every client.

Q2 Redis vs Memcached?

Memcached is genuinely faster for pure string GET/SET at high concurrency (multi-threaded, leaner per-item memory). Redis wins on everything else: rich data structures that make operations atomic server-side, persistence, replication, Sentinel/Cluster, Pub/Sub and Streams. Default to Redis; choose Memcached only for a simple ephemeral string cache where you want the least possible machinery.

Q3 Explain Redis eviction policies. Which would you set?

`allkeys-lru` for a general cache. `allkeys-lfu` if traffic has viral spikes I don't want polluting the hot set. `volatile-lru` when one instance holds both cache data (with TTLs) and persistent data. Critically: the default is `noeviction`, which makes writes fail with OOM once memory fills — correct for a datastore, catastrophic for a cache.

Bonus: Redis approximates LRU by sampling keys into an eviction pool rather than maintaining a global list — and LFU uses an 8-bit Morris counter with time decay.

Q4 How does Redis Cluster shard data?

16,384 hash slots; slot = CRC16(key) mod 16384; each primary owns a slot range. Clients cache the slot map and route directly — no proxy. Rebalancing moves whole slots, so key→slot never changes, only slot→node. During migration you get `MOVED` (permanent, update your map) or `ASK` (temporary, this key only). Multi-key ops require same-slot keys — use `{hash tags}` to co-locate deliberately.

Q5 RDB or AOF?

Both. RDB is a compact point-in-time snapshot — fast restart and ideal for backups, but you lose everything since the last save. AOF logs every write command — with `everysec` fsync you lose at most one second, at the cost of larger files and slower restarts. The hybrid (`aof-use-rdb-preamble`, default since 4.0) gives you snapshot-speed restarts with a one-second loss window.

Nuance: even a pure cache benefits from RDB — it comes back warm after a restart instead of cold-starting your database into an avalanche.

Q6 Is Redis strongly consistent? Can it lose writes?

No, and yes. Replication is asynchronous: a primary can acknowledge a write, fail before replicating, and have a replica promoted that never saw it — the acknowledged write is silently lost. `WAIT` improves the odds but is not consensus. Redis chooses availability and latency. Fine for a cache; be deliberate before treating it as a source of truth. AWS MemoryDB is the durable variant if you need that.

Q7 Sentinel or Cluster — which would you run?

Sentinel is high availability for a single primary: an odd number of monitor processes agree by quorum that the primary is down, elect a replica and rewrite the topology. One dataset, no sharding, ordinary clients — the answer when the working set fits one node and you only need failover.

Cluster is sharding *plus* failover: slots are spread over primaries, each with its own replicas, and the client is cluster-aware. The cost is that multi-key operations, transactions and Lua scripts must touch one slot, so keys have to be designed with hash tags.

Signal: choosing Sentinel and saying why. Reaching for Cluster when one node would do adds routing rules and key-design constraints for no benefit.

Q8 How do keys actually expire?

Two mechanisms. **Lazily**, on access: a read of an expired key deletes it and returns nothing. And **actively**: a cycle runs ten times a second, samples 20 keys with a TTL, deletes the expired ones and repeats while more than a quarter of the sample was expired. So an expired key that nobody reads can occupy memory for a while — which is why `TTL` and memory usage are not the same question.

The detail that scores: replicas do not expire keys on their own clock — they wait for the primary's `DEL` so a read on a replica can briefly see a logically expired key.

Q9 Implement a distributed lock with Redis.

`SET key token NX PX 10000` — atomic acquire with expiry in one command (never `SETNX` then `EXPIRE`). Release via Lua compare-and-delete so you can't delete someone else's lock.

The senior answer: distinguish *efficiency* locks (worst case is duplicate work — Redis is fine) from *correctness* locks (worst case is corruption — use etcd/ZooKeeper, or a DB constraint that gives you fencing). Cite Kleppmann on Redlock's lack of fencing tokens.

Q10 Design a rate limiter.

Walk the five in order, each fixing the last one's flaw: **fixed window** (one counter, but allows 2× the rate across the boundary) → **sliding log** (a ZSET of timestamps, exact but one entry per request) → **sliding window counter** (two counters weighted by overlap — O(1) memory, ~99% accurate, the general-purpose default) → **token bucket** (lazy refill in a Hash; allows a burst if you've been idle — the usual public-API choice) → **leaky bucket** (same shape, drains at a fixed rate, never allows bursts — for protecting a fragile downstream).

All of it runs in Lua, because a rate check is read-then-write and must be atomic.

Follow-ups to pre-empt: multiple servers → the counter must be central (or two-tier); Redis down → decide fail-open vs fail-closed; response → HTTP 429 with `Retry-After` and `X-RateLimit-*` headers.

Q11 Build a real-time leaderboard for 50 million players.

Sorted set: `ZADD` / `ZINCRBY` to update (O(log N)), `ZREVRANGE 0 9` for the top 10, `ZREVRANK` for a player's exact rank in O(log N) — which SQL cannot do without a full scan. Time-windowed boards are separate ZSETs with TTLs, combined via `ZUNIONSTORE`. Break ties by packing a timestamp into the score's low bits.

Q12 Your p99 latency spikes randomly but p50 is fine. Redis looks healthy. Why?

Almost always the single-threaded event loop being blocked. Suspects: a big key (`HGETALL` on a huge hash, `DEL` on a multi-MB value), a `KEYS *` in some forgotten admin script, a slow Lua script, or an fsync/fork() pause from persistence. Diagnose with `SLOWLOG GET`, `--bigkeys`, `--latency-history`. Fix by splitting keys, using SCAN-family commands, and `UNLINK` instead of `DEL`.

Q13 Pipelining, MULTI/EXEC or Lua — when do you use which?

Pipelining is a network optimisation: send N independent commands without waiting for each reply. No atomicity — other clients interleave.

MULTI/EXEC queues commands and runs them with nothing interleaved, but there is no rollback and no way to branch on a value you read inside the transaction. `WATCH` adds optimistic locking: abort if a watched key changed.

Lua (or a Function) is the only one that can read a value and decide what to write in the same atomic step — which is why every rate limiter and every compare-and-delete release is a script.

Say the rule: independent commands → pipeline; "all or nothing" → MULTI; read-then-write → Lua.

Q14 What is a Bloom filter and where does it fit here?

A probabilistic set: definite "not present," probabilistic "present." k hash functions set k bits per element. Because there are no false negatives, it's safe to reject on. It gates cache penetration: hold all valid IDs, and reject non-existent ones before they touch Redis or the database. ~1.2 GB for a billion IDs at 1% false-positive rate. Can't delete — use a Cuckoo filter if IDs are removed. In Redis it is `BF.ADD` / `BF.EXISTS` from the Bloom module.

Q15 How do you keep Redis memory under control?

Four levers, in the order I'd apply them. **Measure the right number:** `used_memory` is what the allocator holds for data, `used_memory_rss` is what the OS sees — a ratio much above 1.5 is fragmentation, not data. **Shrink the encoding:** many small hashes stay listpack-encoded and cost a fraction of the same fields as top-level keys — this is the Instagram result. **Bound it:** set `maxmemory` with an eviction policy so the instance sheds instead of failing writes. **Hunt the outliers:** `--bigkeys` for the biggest key per type, then split it and free it with `UNLINK`.

3.4 Red flags interviewers listen for

Saying this	Says this about you
"Redis is strongly consistent"	Factually wrong — replication is async
Using `KEYS *` in a design	Immediate red flag to anyone who has run Redis
Reciting Redlock as the distributed-lock answer	Missing the well-known critique and the efficiency/correctness distinction
`SETNX` then `EXPIRE` for a lock	A crash between the two leaves a lock nobody can release
"Add more shards" as the answer to a hot key	One key maps to one slot; slots do not split
Leaving the `noeviction` default on a cache	Writes start failing with OOM instead of evicting
`HGETALL` or `SMEMBERS` on a huge key in a hot path	Blocks the event loop for every other client
Treating Pub/Sub as a queue	No persistence, no replay, no acknowledgement — that is Streams
"AOF everysec, so it's durable"	fsync is not consensus; a failover still loses acknowledged writes
Never mentioning the single thread	The fact that explains most Redis incidents

3.5 Real systems worth name-dropping

Instagram — Redis memory

Bucketing IDs into thousands of small hashes (listpack-encoded) instead of millions of top-level keys cut memory roughly 5×. Cite it for encodings and per-key overhead.

Twitter/X — timelines

Precomputed timelines held as Redis lists, with hybrid fan-out for celebrities. The canonical example of Redis as a serving structure, not a cache.

Kleppmann vs Sanfilippo on Redlock

The 2016 exchange on whether a Redis lock is safe without fencing tokens. Cite both sides: it is the fastest way to show you know where the boundary is.

MemoryDB and the Valkey fork

AWS MemoryDB is the durable, multi-AZ variant when you need a Redis API with consensus; Valkey is the 2024 BSD fork after the licence change. Ecosystem awareness, cheaply earned.

Redis commands you're expected to know

REFERENCE

Not the full 240-command manual — the subset that someone who has actually worked with Redis would use without looking up. Learn the **bold** ones cold; recognise the rest. **O(1)** means safe at any size; **O(N)** means it can block the single thread, so check the size first.

Keys, TTL and the basics

Command	Cost	What it does & why you'd reach for it
SET k v	O(1)	Store a value. The options are the real content — see the table below.
GET k	O(1)	Read a value. Returns nil if absent.
DEL k [k…]	O(N)	Delete. **O(N) in the size of the value** — deleting a huge collection blocks the server.
UNLINK k	O(1)	Delete, but free the memory on a background thread. **Use this instead of DEL for big keys.**
EXPIRE k sec	O(1)	Attach a TTL. `PEXPIRE` for milliseconds, `EXPIREAT` for an absolute time.
TTL k	O(1)	Seconds remaining. **−1** = no TTL set, **−2** = key doesn't exist. Knowing those two values is a small credibility marker.
PERSIST k	O(1)	Remove the TTL, making the key permanent.
EXISTS k	O(1)	Does it exist. With multiple keys it returns a *count*, not a boolean.
TYPE k	O(1)	string / hash / list / set / zset / stream. Useful when debugging a mixed keyspace.
SCAN cursor	O(1)/call	**The safe way to iterate.** Cursor-based, non-blocking. `MATCH` to filter, `COUNT` to hint batch size. May return duplicates — dedupe client-side.
KEYS pattern	O(N)	**Never run this in production.** Scans the entire keyspace and blocks everything. Its presence in a design is an instant red flag.

SET options — the ones that carry real meaning

Form	Meaning
SET k v EX 60	Set with a 60-second TTL. `PX` for milliseconds. Equivalent to the older `SETEX`.
SET k v NX	Only if the key does *not* exist. This is the atomic "claim it" primitive.
SET k v XX	Only if the key *does* already exist.
SET k v NX PX 10000	**The distributed-lock idiom.** Claim-if-free plus expiry, atomically, in one command. Never do `SETNX` then `EXPIRE` — a crash between them leaves a lock nobody can release.
SET k v KEEPTTL	Overwrite the value but keep the existing TTL running.

Strings & counters

Command	Cost	Notes
INCR / DECR k	O(1)	Atomic ±1. The foundation of counters and fixed-window rate limiting. No race conditions, ever.
INCRBY k n	O(1)	Atomic add. `INCRBYFLOAT` for decimals. Lets you batch 1,000 increments into one call.
MGET / MSET	O(N)	Many keys in **one round trip**. This is how you read a sharded counter without 16 network hops. In Cluster, all keys must share a slot.
GETDEL k	O(1)	Read and delete atomically — one-shot tokens, OTPs.
SETRANGE / GETRANGE	O(N)	Operate on part of a string without transferring the whole thing.
STRLEN k	O(1)	Length. Cheap way to check whether a value has grown into a big key.

Hashes — for objects

Command	Cost	Notes
HSET k f v	O(1)	Set one field. Accepts several field/value pairs at once.
HGET k f	O(1)	**Read one field without deserialising the whole object** — the main reason to use a Hash over a JSON String.
HGETALL k	O(N)	Every field. Fine for a 10-field session, dangerous on a hash with a million fields.
HINCRBY k f n	O(1)	Atomic counter *inside* a hash. This is what makes the token-bucket rate limiter possible.
HDEL / HEXISTS / HLEN	O(1)	Remove a field / test a field / count fields.
HSCAN k cursor	O(1)/call	Safe iteration over a large hash. The alternative to a blocking `HGETALL`.

Lists — for queues and feeds

Command	Cost	Notes
LPUSH / RPUSH	O(1)	Push to head / tail.
LPOP / RPOP	O(1)	Pop from head / tail. `LPUSH` + `RPOP` = FIFO queue.
LRANGE k 0 9	O(N)	Read a range. `LRANGE k 0 -1` reads everything — the classic accidental big-key read.
LTRIM k 0 799	O(N)	**Keep only the newest 800 entries.** LPUSH + LTRIM is *the* capped-timeline idiom.
BLPOP k timeout	O(1)	Blocking pop — a worker waits instead of polling. `0` means wait forever.
BLMOVE src dst	O(1)	Atomically move to a "processing" list while popping, so a crashed worker's job isn't lost. The reliable-queue pattern.
LLEN k	O(1)	Length — your queue-depth metric.

Sets — for membership

Command	Cost	Notes
SADD / SREM	O(1)	Add / remove. `SADD` returns 0 if it was already there — a free dedupe test.
SISMEMBER k v	O(1)	Is it in the set. The tag / permission / blocklist check.
SCARD k	O(1)	Exact count of unique members.
SMEMBERS k	O(N)	Every member. Same big-key hazard as `HGETALL` — prefer `SSCAN`.
SINTER / SUNION / SDIFF	O(N×M)	Server-side set algebra — mutual friends, common tags. Powerful but genuinely expensive; consider `SINTERCARD` if you only need the size.
SPOP / SRANDMEMBER	O(1)	Remove-and-return random / peek random. Raffles, sampling, random work stealing.

Sorted sets — the most important type

Command	Cost	Notes
ZADD k score m	O(log N)	Insert or update a member's score.
ZINCRBY k n m	O(log N)	Atomically bump a score — the leaderboard update.
ZRANGE k 0 9 REV	O(log N + M)	Top 10. Add `WITHSCORES` to get the values too. (`ZREVRANGE` is the older spelling.)
ZRANK / ZREVRANK	O(log N)	**A member's exact position among millions, in log time.** This single command is why leaderboards live in Redis rather than SQL.
ZRANGEBYSCORE	O(log N + M)	Everything in a score band. With a timestamp as the score this is a time-range query — the delayed-job queue.
ZREMRANGEBYSCORE	O(log N + M)	Delete a score band. **The sliding-window rate limiter's trim step.**
ZCARD / ZCOUNT / ZSCORE	O(1) / O(log N)	Total members / members in a band / one member's score.
ZPOPMIN / ZPOPMAX	O(log N)	Pop the lowest / highest. Turns a sorted set into a priority queue.
ZUNIONSTORE / ZINTERSTORE	O(N)	Merge boards — combine daily leaderboards into a monthly one.

Probabilistic & specialised

Command	Notes
PFADD / PFCOUNT	HyperLogLog. **Count billions of unique items in 12 KB** with ~0.81% error. `PFMERGE` rolls daily uniques into a monthly figure. You cannot list the members back.
SETBIT / BITCOUNT	Bitmap. One bit per user → 10 M users of daily-active tracking in 1.2 MB. `BITOP AND` across two days gives you a retention cohort.
GEOADD / **GEOSEARCH**	"Everything within 3 km of here, nearest first." Backed by a sorted set of geohashes.
XADD / XREADGROUP / XACK	Streams. Append an event / read as part of a consumer group / acknowledge it. `XAUTOCLAIM` recovers work from a dead consumer.
BF.ADD / BF.EXISTS	Bloom filter (RedisBloom module) — the cache-penetration gate.

Transactions, scripting, pipelining

Command	Notes
MULTI / EXEC	Queue commands and run them with nothing interleaved. **There is no rollback** — if one command fails the others still apply.
WATCH k	Optimistic lock: `EXEC` aborts if the watched key changed. Requires a client-side retry loop.
EVAL script	Run Lua atomically on the server. The answer to any read-then-write that must be indivisible — every rate limiter in Part 2.
EVALSHA / SCRIPT LOAD	Send the script's hash instead of its body. What clients do automatically after the first call.
Pipelining	Not a command — a client feature. Send N commands without waiting for each reply. **Often 5–10× throughput, but not atomic.** Interviewers like this distinction.

Operations & debugging — the experience tell

Command	What it tells you
INFO [section]	Everything: `INFO memory`, `INFO stats` (hits/misses/evictions), `INFO replication`. Your first move on any incident.
SLOWLOG GET	Recent commands that exceeded the threshold. **The first place to look for p99 spikes.**
OBJECT ENCODING k	listpack vs hashtable vs skiplist — proves whether a small-object optimisation is actually in effect.
OBJECT FREQ / IDLETIME	LFU access counter / seconds since last touched. Requires the matching eviction policy.
MEMORY USAGE k	Exact bytes for one key. `MEMORY DOCTOR` gives a plain-English diagnosis.
CONFIG GET/SET	Read and change settings live — `maxmemory`, `maxmemory-policy`, `appendfsync`.
CLIENT LIST / KILL	Who is connected; drop a misbehaving one. Diagnose connection leaks.
CLUSTER INFO / NODES	Cluster health and slot ownership. `CLUSTER KEYSLOT k` shows which slot a key hashes to — how you prove a hash-tag theory.
BGSAVE / BGREWRITEAOF	Snapshot / compact the AOF in the background. `LASTSAVE` confirms it worked.
REPLICAOF host port	Make this node a replica. `REPLICAOF NO ONE` promotes it.
MONITOR	Streams every command live. Superb for debugging, **severe throughput cost** — never leave it running.
FLUSHALL / **FLUSHDB**	Wipe everything. Add `ASYNC` so it doesn't block. Usually disabled by rename in production.

CLI flags worth knowing by name

--bigkeys	Samples the keyspace and reports the largest key of each type. Your big-key hunt.
--hotkeys	Finds the most frequently accessed keys (needs an LFU policy). Your celebrity-key hunt.
--memkeys	Like `--bigkeys` but ranked by actual memory rather than element count.
--latency / --latency-history	Measures round-trip latency continuously — distinguishes "Redis is slow" from "the network is slow."
--stat	A live one-line-per-second dashboard: ops/sec, memory, clients, keys.
--scan	Safely dumps all key names using SCAN under the hood.

THE FIVE COMMANDS THAT MARK YOU AS DANGEROUS

`KEYS *` · `FLUSHALL` without `ASYNC` · `MONITOR` left running · `HGETALL` / `SMEMBERS` / `LRANGE 0 -1` on an unbounded collection · `DEL` on a multi-megabyte key.

All five share one cause: **Redis executes commands on a single thread**, so any of them freezes every other client for its full duration. If you name that as the common thread rather than listing the commands, you've answered the question behind the question.

THE SIX THAT MARK YOU AS EXPERIENCED

UNLINK instead of DEL · **SCAN** instead of KEYS · **SET k v NX PX** as one atomic lock acquisition · **LPUSH + LTRIM** for a capped feed · **OBJECT ENCODING** to verify a memory optimisation · **SLOWLOG** as the first stop on a latency incident.

REVISION

One-page cheat sheet

The night-before page. If you remember only this, you can still hold a good conversation about Redis.

Redis in ten facts

Single-threaded execution

Every command atomic; one slow command blocks everyone. Socket I/O is threaded since 6.0.

Async replication

Can lose acknowledged writes on failover. Not CP. Fine for a cache, deliberate for anything else.

16,384 hash slots

CRC16 mod 16384. Rebalancing moves slots, not keys. `{tags}` co-locate.

Approximated LRU/LFU

Samples into an eviction pool; LFU uses an 8-bit Morris counter with decay.

Lazy + active expiry

On access, plus 20 random samples 10×/sec. Expired keys can linger; replicas wait for the primary.

RDB + AOF hybrid

Snapshot preamble + command tail. Fast restart, ≤1 s loss with everysec.

Encodings matter

listpack → hashtable/skiplist at thresholds. Many small hashes beat one giant one.

Sorted sets are the star

O(log N) rank. Leaderboards, sliding windows, priority queues, geo.

Lua is the atomicity tool

The only way to read a value and decide the write in one indivisible step.

RSS is not data

used_memory vs used_memory_rss: a ratio well above 1.5 is fragmentation, not growth.

The default decisions

Question	Default answer
Cache or datastore?	Cache first. Treating Redis as the source of truth needs a durability argument
Persistence?	RDB + AOF hybrid; everysec fsync
Eviction policy?	allkeys-lru · allkeys-lfu when viral spikes pollute the hot set; never leave noeviction
HA shape?	Sentinel while one node holds the data; Cluster once it does not
Lock?	SET NX PX + Lua release for efficiency; a consensus store or DB constraint for correctness
Queue?	Streams with consumer groups. Pub/Sub only for fire-and-forget
Counter at scale?	INCR, sharded if hot; HyperLogLog when approximate is fine
Rate limiter?	Sliding window counter in Lua; token bucket for public APIs
Read-then-write?	Lua, always. Never a client-side round trip

The failure profile

Symptom	Mechanism	What you do
p99 spikes, p50 flat	Event loop blocked by one slow command	SLOWLOG, --bigkeys; split keys; UNLINK
Acknowledged write vanished	Async replication + failover	WAIT, or a store with consensus
One node hot, rest idle	A single key or slot takes the traffic	Local L1 cache; split the key; never a hash tag
Writes fail with OOM	maxmemory reached under noeviction	Set an eviction policy; size the working set
Periodic latency plateau	fork() for RDB/AOF rewrite copies page tables	Off-peak saves; smaller instances; disable THP
Memory grows without data	Fragmentation or lingering expired keys	Check RSS ratio; activedefrag; review TTLs

IF YOU SAY NOTHING ELSE, SAY THIS

"Redis is a single-threaded, in-memory data-structure server. That buys me server-side atomic operations — counters, sorted sets, Lua — and it costs me one shared thread, so every command has to be O(1) or O(log N). Replication is asynchronous, so an acknowledged write can be lost on failover: I'd use it as a cache and a coordination primitive, and reach for a consensus store when correctness depends on the lock."

Sources & further reading — Redis official docs (data types, expiry, eviction, persistence, replication, Sentinel, Cluster specification, client-side caching, Functions, Streams); Redis blog: *LFU vs LRU*, *How to tame the thundering herd*; Instagram Engineering, *Storing hundreds of millions of simple key-value pairs in Redis*; Martin Kleppmann, *How to do distributed locking* (2016) and Salvatore Sanfilippo's rebuttal; Alibaba Cloud, *Redis Hotspot Key Discovery and Common Solutions*; AWS docs on ElastiCache and MemoryDB; the Valkey project; Alex Xu / ByteByteGo and Hello Interview Redis primers.